FunnelStream

Team handoff: questions and answers

A production-style streaming and batch data platform for an Egyptian online store: Kafka, Spark, Delta Lake, Databricks, Postgres, Grafana, Airflow and Power BI.

State of the project: **8 October 2026, branch funnelstream-build, commit f27bd14**.
Defense: **13 October 2026**. Team: 5 members.

How to use this document: Part 1 explains what exists and why. Part 2 shows how to run it. Part 3 lists the remaining tasks with owners and acceptance checks. Part 4 covers decisions, limits and troubleshooting. The repository is the source of truth: the spec (`docs/superpowers/specs/`), the plan (`docs/superpowers/plans/`), the run guide (`docs/RUNNING.md`) and the metric definitions (`docs/metrics.md`) go deeper than this summary.

Contents

- **Part 1. What it is and how it works**: the project, architecture, data generation, Bronze, Silver, Gold, attribution, live monitoring, the ad report.
- **Part 2. Running it**: setup, the live demo, loading Gold, tests, secrets, memory.
- **Part 3. Remaining tasks**: every open task with owner, files, steps and acceptance check; what to cut if time runs out.
- **Part 4. Decisions, limits and troubleshooting**.
- **Appendix**: repository map, status table, vocabulary.

Status at a glance

Area	State	Evidence
Generator, noise, mock ad API	Done	19 noise kinds with exact manifest counts; 39 official districts under 6 cities; tracking URL with UTM
Kafka, Bronze, Silver	Done	3 topics; idempotent; Silver counts reconcile to the manifest to the row
Databricks mirror and Gold build	Done	Real workspace: 0 mismatches vs the Python reference on all Gold tables built so far
Attribution (T13)	Done	fact_attribution 702 rows for 351 orders, 0 mismatches on the real workspace
Live monitoring (G1, G2)	Done	Postgres live tables, Grafana dashboard (14 panels), 3 alert rules fired in a live check
Ad report DAG	Done	Airflow dags test: first run pulled, second run cached, identical numbers
Tests	Green	Host 645 passed, 121 skipped. Full Docker suite 765 passed, 1 skipped (25 min)
Open work	8 tasks	T12, T14, T15, T16, G3, T17, T18, DOC (+ stretch T19). See Part 3

Part 1. What it is and how it works

The project

What is FunnelStream and what problem does it solve?

A simulated online shop in Egypt. Customers browse in an app or on the web, add to cart, check out (mostly cash on delivery, some prepaid), and orders are delivered, sometimes failing or being returned. The platform ingests that traffic as a stream, cleans it, builds business tables in Databricks and shows live pipeline health in Grafana. It is built the way a production platform would be, not as a classroom ETL: dirty data on purpose, late and duplicate events, idempotent writes, alerts, and a documented rule for every number.

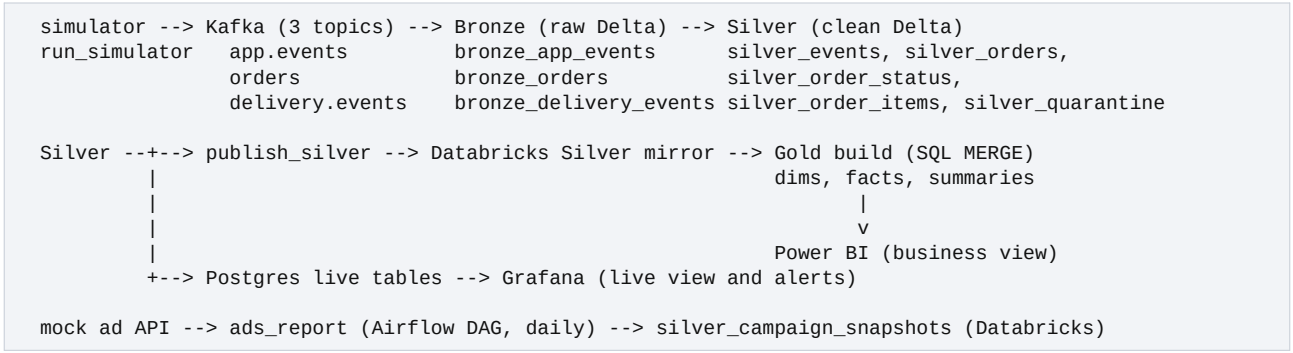
Which business questions does it answer?

- Where do users drop in the funnel (by day, channel and platform: app or web)?
- Which campaigns bring orders, at what cost per order and per delivered order (last-touch attribution)?
- How many orders are delivered, cancelled, returned, or had at least one failed delivery attempt? (Failed cash-on-delivery is the main story.)
- Which products do people view and buy per city, and what should we recommend next?
- Is the pipeline healthy right now (events per minute, late events, quarantined rows, freshness)?

Why is the data synthetic, and is it realistic?

No access to a real shop. A Python generator with a fixed seed produces journeys and orders with realistic drop-off, payment mix, delivery failures and Egyptian cities and districts. It also injects defects on purpose and writes `noise_manifest.json` with the exact count of each, so the pipeline can be proven correct: quarantine plus duplicates must match the manifest. The limit is stated in the spec: synthetic data cannot be proven 100% realistic.

Architecture



Which technology does what, and why?

Component	Role	Why this choice
Kafka (1 broker)	Event bus: app.events, orders, delivery.events	Standard streaming backbone; keys keep one session or order in order
Spark (in Docker)	Bronze ingest, Silver clean, local reference builders	Structured Streaming + Delta MERGE; Docker because Spark needs winutils on Windows
Delta Lake (local)	Bronze and Silver storage	ACID, MERGE for idempotent writes, no cloud needed locally
Databricks	Silver mirror and all Gold tables	SQL MERGE builds, shared by the team, Power BI connector
Postgres	Live and health tables for Grafana; DDL fallback for Gold	Light, Grafana reads it directly
Grafana	Live dashboard (5 s refresh) and 3 alert rules	Light on memory; Power BI is for business analysis, Grafana for live monitoring
Airflow	Daily ad report DAG	Scheduling; off by default (2 GB), started with a compose profile
Power BI	Business dashboards (open task T18)	Connects to the Databricks Gold tables

Why is Gold written by Databricks only?

One writer per table (spec section 17, rule 1). Late and corrected data is fixed by recomputing the affected dates from Silver and MERGE-ing the result, never by adding increments. The local Spark job writes only Bronze, Silver and the Postgres live tables.

Data generation

What generates the data and what does one run produce?

`scripts/run_simulator.py` (modules in `simulator/`). One run publishes three kinds of messages to Kafka: app events (browsing and purchase), order lifecycle events and delivery events. An order exists only for a session that reached *purchase*. Example run (seed 7, 20,000 events): 19,316 app events, 667 order events, 383 delivery events, 20,366 messages in total.

What are the order and delivery rules?

- Status rank (highest wins): order_created 1, payment_authorised 2, order_packed 3, order_shipped 4, out_for_delivery and delivery_failed 5, delivered 6, order_completed and order_returned 7 (mutually exclusive; a tie goes to the latest status time). order_cancelled has rank 2 and counts only before shipment.
- **Cash on delivery skips payment_authorised**; payment is collected at delivery. Prepaid methods (card, fawry, vodafone_cash, instapay) emit it.
- A delivery can fail and be retried. If the last attempt fails and none succeeded, the order ends as **order_returned**.
- Current status is the highest rank by **event time**, never by arrival order. A cancel counts only if it happened before the shipment; a cancel after shipping is quarantined (CANCEL_AFTER_SHIPMENT).

What defects are injected (noise)?

19 kinds, each counted exactly in the manifest: duplicates, malformed JSON, missing fields, unknown event types, bad versions, negative prices, future timestamps, late and very late events, non-UTC timestamps, null and unknown product ids, dirty text (city, device), numbers sent as strings, out-of-order status, cancel after shipment, bot sessions, **dirty districts** and **dirty UTM values**. Defects the validator can see go to quarantine; the others are cleaned in Silver.

What is an address and what is the tracking URL?

- Address = country > city > **district**. Cities: Cairo, Giza, Alexandria, Mansoura, Tanta, Assiut, Other. 39 official districts (qisms and ahya) under the six named cities, plus the single value Other, for example Nasr City, Maadi and Heliopolis under Cairo; Dokki, Agouza and Imbaba under Giza. Source of truth: `contracts/geo.py`.
- A missing, unknown or mismatched district is stored as **unknown** and the row is kept (never quarantined). **Other** is a real value for smaller cities. Mohandessin and Faisal are neighborhoods, not districts, so they are not used.
- Campaign traffic is tracked by UTM parameters inside `page_url` on the landing and product-view events, for example `https://shop.example.eg/p/prod_0042?utm_source=meta&utm_medium=paid_social&utm_campaign=ramadan_offer_1`. Silver parses utm_source, utm_medium and utm_campaign from it.

What is the difference between --seed and --universe-seed?

--universe-seed fixes the customers and the product catalog (always 7 in this project). **--seed** fixes the events: session, order and event ids. The same seed regenerates the same ids and Silver silently drops them as duplicates, so every new run needs a **new --seed** with the same --universe-seed. `watch_demo.ps1` keeps the next free seed in `data/state/next_seed.txt`.

Kafka, Bronze and Silver

What are the Kafka topics?

`app.events` (6 partitions, key session_id), `orders` (3, key order_id) and `delivery.events` (3, key order_id). There are no dead-letter topics: bad messages go to the Silver quarantine table (one term, one place).

What does Bronze do?

`streaming/jobs/bronze_ingest.py`: one streaming query per topic writes every raw message, untouched, to a Delta table with Kafka partition, offset, timestamp and ingest time. Nothing is dropped or parsed. Processing is at-least-once; a restart resumes from the checkpoint.

What does Silver do?

- Validates (schema, types, version, product exists), normalizes city, device, district, timestamps (UTC) and numbers, and parses UTM from page_url.
- Deduplicates by event id with an idempotent Delta MERGE, partitioned by Cairo event date.
- Flags late events (`is_late`, more than 10 minutes behind) and bot sessions (more than 100 events in 10 seconds).
- Writes silver_events, silver_orders, silver_order_status, silver_order_items and silver_quarantine. Products and customers are seeded first (`scripts/seed_silver_reference.py`).
- Never raises on bad input: a bad row becomes a quarantine row with an error code and detail, never the raw payload.

What are the quarantine error codes?

BAD_JSON, MISSING_FIELD, UNKNOWN_EVENT_TYPE, BAD_VERSION, NEGATIVE_PRICE, FUTURE_TIMESTAMP, BAD_VALUE, UNKNOWN_PRODUCT, CANCEL_AFTER_SHIPMENT, NEGATIVE_SPEND, CLICKS_GT_IMPRESSIONS.

How do we know Silver is correct?

It reconciles to the manifest. Example (seed 7): 20,366 messages - 314 quarantined - 199 duplicates = 19,853 = 18,822 silver_events + 1,031 silver_order_status rows. The 314 equals the sum of the manifest counts for the defects the validator can see.

How are late events handled?

Late rows stay in Silver and in Gold (funnel, attribution, orders). They are excluded only from the live metrics. In history mode (old events replayed) every row is late by the rule, so the late-rate alert is meaningful for `--live` runs only.

Databricks Silver mirror and Gold

How does data get from local Silver to Databricks?

`scripts/publish_silver.py` uploads an explicit allow-list of columns (no names, phones or raw payloads) as Parquet to a Databricks volume and MERGEs insert-only into the Silver mirror tables. It is idempotent. `--reset --yes` truncates the mirror tables first (needed after a fresh local run, because the mirror only adds rows).

Which Gold tables exist?

Table	Meaning	Built by
dim_customer, dim_product	Versioned (SCD2) dimensions; key -1 = unknown member	Gold build (done)
dim_campaign, dim_date	3 campaigns; calendar 2025-2027 with weekend, Ramadan, Eid, holidays	Gold build (done)
fact_orders, fact_order_lines	One row per order / order line; current status, flags, district, platform, campaign_id	Gold build (done)
fact_attribution	Two rows per order: last_touch and first_touch	Gold build (done, T13)
agg_orders_daily	Orders and revenue by day, channel, payment method, platform	Gold build (done)
agg_funnel_daily	Sessions and drop-off by day, country, channel, platform	Gold build (done)
agg_campaign_daily	Spend, orders, CPA, cost per delivered order, rates per campaign and date	**T12 (open)**
agg_city_product_daily	Views, carts, sales per city, product, day	**T14 (open)**
rec_product, customer_360	Recommendations; customer current state	T14 / T19 (cut first)

How does the Gold build run?

```
docker compose run --rm spark python -m warehouse.gold_build --days 7      # last 7 days (default)
docker compose run --rm spark python -m warehouse.gold_build --full        # rebuild everything
docker compose run --rm spark python -m warehouse.gold_build --days 7 --dry-run    # prints the SQL only
```

Steps run in a fixed order: dimensions, dim_date, fact_orders, fact_order_lines, fact_attribution, agg_orders_daily, agg_funnel_daily, then the slots for the open tasks. Each step is a MERGE script in `warehouse/gold/NN_name.sql`. A failing step stops the run and names itself. The session time zone is pinned to UTC; dates use Africa/Cairo.

What is the unknown member (key -1)?

A fact whose customer or product has no dimension row yet (a brand-new customer or product, or an anonymous customer) stores key -1 instead of being quarantined. Facts keep the natural ids, so the next Gold build re-joins and repairs the key once the dimension row exists.

What is the limit of the 7-day window?

A late status row for an order older than the window is missed until a wider `--days` or a `--full` run. Run `--full` now and then (the funnel is not affected).

How is attribution decided?

- **Click** = the first page_view or product_view in a session that carries utm_campaign; bot sessions and unresolved campaigns do not count.
- **last_touch**: the order's own session click (click_id on the order) first; else the customer's latest click in the 7 days before the order (identified customers only); else **direct** (campaign NULL).
- **first_touch**: own session click; else the customer's earliest click in the same 7-day window; else direct.
- agg_campaign_daily uses last_touch and says so in an attribution_model column. direct_share is computed from agg_orders_daily (channel direct), not per campaign.
- Linking an anonymous pre-login session to a customer after login is out of scope (documented limitation).

What are the business definitions?

Metric	Definition
Placed	Every attributed order, whatever its final status
failed_attempt_rate	Orders with at least one delivery_failed / orders placed
return_rate	Orders whose final status is order_returned / orders placed
open_rate	Orders not delivered, cancelled or returned after 14 days / orders placed
CPA / cost per delivered order	spend / orders placed; spend / orders delivered (both NULL when the denominator is 0)
Revenue dating	Delivered revenue is counted on the ORDER date (cohort view), so recent days are incomplete
CTR	Clicks reported by the ad API / impressions (not the same clicks as attribution)

Live monitoring and the ad report

What does Grafana show and where does the data come from?

The Silver job, after each batch, recomputes full per-minute totals and upserts them (`INSERT ... ON CONFLICT DO UPDATE`, never +1) into four Postgres tables: `live_metrics_minute` (events, active sessions, product views, add to cart, orders per minute and country), `pipeline_health_minute` (events, late events, quarantined rows, Silver batches), `pipeline_batch_log` and `pipeline_heartbeat` (silver_job, gold_build, publish: when each last reported). Grafana (http://localhost:3000, 5 s refresh, anonymous view-only) reads them with a read-only role. Panels: events per minute, active sessions, orders, late-event rate, quarantine share and rows per hour, seconds since the last Silver report, Gold update and publish.

Which alerts exist?

Rule	Fires when	Note
Late-event rate	Above 5% over 5 minutes (at least 20 events)	Meaningful for --live runs only

Rule	Fires when	Note
Quarantine share	Above 2% over 5 minutes (at least 20 rows)	quarantined / (events + quarantined)
No events	No events for 2 minutes (no data counts as firing)	Fires when the generator is idle

Alerts go to a webhook. `scripts/alert_sink.py` receives them, prints one line each in the terminal and logs them to `logs/alerts.log`.

What if Postgres is down?

The Silver job still commits Silver first; the live-store write is best effort with timeouts, and the pending minutes are repaired by the next successful batch. Silver is always the source of truth.

What is the daily ad report?

Airflow DAG `ads_yesterday_report` (03:00 UTC, logic in `batch/ads_report.py`) pulls the mock ad API for yesterday once, stores it in `silver_campaign_snapshots` and writes `data/reports/ads_report_YYYY-MM-DD.json`. A rerun on the same day returns the stored report without calling the API. Bad rows (negative spend, clicks above impressions) go to quarantine. It needs `GOLD_BACKEND=databricks`.

Part 2. Running it

What do I need installed?

Docker Desktop (running, 8 GB or more), Python 3.11, Git, and a Databricks workspace with a SQL warehouse and your own personal access token. Java is NOT needed on the host.

How do I set up on a new machine?

```
git clone https://github.com/omarmohosny/funnelstream.git
cd funnelstream
git checkout funnelstream-build        # or main once the pull request is merged
python -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt
Copy-Item .env.example .env            # then fill in the values listed below
docker compose build spark
.venv\Scripts\python -m pytest -q      # expect: 645 passed, 121 skipped
```

Values to set in `.env`: `DATABRICKS_HOST`, `DATABRICKS_TOKEN`, `DATABRICKS_WAREHOUSE_ID`, `DATABRICKS_CATALOG`, `POSTGRES_PASSWORD`, `GRAFANA_DB_PASSWORD`, `GRAFANA_ADMIN_PASSWORD`. The file is git-ignored: never commit it and never paste tokens in chat.

How do I run the live demo?

```
# opens the Kafka listener, the Bronze+Silver pipeline and the generator; starts Postgres and Grafana
.\scripts\watch_demo.ps1
# stops the pipeline container; then close the windows
.\scripts\stop_demo.ps1
```

Wait about a minute for Spark to start; the generator starts after that. Open http://localhost:3000 to watch the lines rise. Options: `-Events 20000 -Rate 50 -ShowEvery 20 -Sample 3`. To see the alerts print, run `.venv\Scripts\python scripts\alert_sink.py` in another window; stop the generator and the 'no events' alert fires after 2 to 3 minutes.

How do I run the pipeline by hand, step by step?

```
docker compose up -d kafka postgres
.venv\Scripts\python kafka/create_topics.py
# once per fresh lake
docker compose run --rm spark python scripts/seed_silver_reference.py --lake data/lake
# generator: new --seed every run, same --universe-seed
.venv\Scripts\python scripts/run_simulator.py --live --rate 50 --events 3000 `
    --seed 22 --universe-seed 7 --show --show-every 10
# Bronze + Silver in one container
docker compose run --rm spark python -m streaming.jobs.pipeline `
    --lake data/lake --checkpoints data/checkpoints --show 3
```

The history mode (no `--live`) loads old days quickly and is what the Gold tables need for a first dataset. Use a new `--seed` each time.

How do I get data into Databricks Gold?

```
docker compose run --rm spark python scripts/check_databricks.py --upload-test
# once: creates the tables and seeds the -1 rows
docker compose run --rm spark python scripts/apply_ddl.py --backend databricks
# add --reset --yes after a fresh local run
docker compose run --rm spark python scripts/publish_silver.py
docker compose run --rm spark python -m warehouse.gold_build --days 7
```

After a schema change use `apply_ddl.py --recreate TABLE,TABLE --yes` (drops and recreates those tables; it prints the plan and exits 2 without --yes), then republish and rebuild.

How do I look at the results?

Databricks SQL editor, for example `SELECT * FROM funnelstream.gold.fact_orders LIMIT 20`, then fact_attribution, agg_orders_daily, agg_funnel_daily. Postgres: `docker compose exec postgres psql -U fs -d funnelstream -c "select * from live_metrics_minute order by minute_start desc limit 10;"`. Local counts: `docker compose run --rm spark python data/count.py data/lake/silver/silver_events` (create data/count.py from docs/RUNNING.md).

How do I run the tests?

```
# host suite, 20-30 s
.venv\Scripts\python -m pytest -q
# full suite, about 25 min
docker compose run --rm -e RUN_SPARK_TESTS=1 spark python -m pytest tests -q
# one Spark test file
docker compose run --rm -e RUN_SPARK_TESTS=1 spark python -m pytest tests/test_gold_build_spark.py -q
```

Run only one Spark container at a time (each can use 4 GB). Expect the full suite: **765 passed, 1 skipped**. Exit code 137 means out of memory: close other containers and rerun.

How much memory does the stack need?

Docker needs about 8 GB: Kafka 1 GB, Postgres 0.5 GB, Grafana 0.5 GB, one Spark container 4 GB (limit and heap are in `docker-compose.yml` and `streaming/common/spark.py`). Airflow (2 GB) is off by default: `docker compose --profile airflow up -d airflow`. Do not run the full test suite and the live pipeline at the same time.

What are the working rules for the team?

- Branch from main, one branch per task (`task-name`), open a pull request, get one review.
- Never commit `.env`, `.venv` or `data/`. Each person uses their own Databricks token.
- Only the Gold build writes Gold. No code outside `warehouse/store.py` talks to a database directly: use `get_store()`.
- A new feature needs a test, and parity against the Python reference where a reference exists. Run the host suite before every push.
- Every simulator run needs a new --seed and --universe-seed 7.

Part 3. Remaining tasks

Eight tasks are open before the 13 October defense, plus one stretch task. Dates are the plan from sprint sheet v3 (`docs/FunnelStream-Sprint-Plan-v3.xlsx`). Owners are the sprint-sheet members; swap them as the team prefers.

ID	Task	Owner	Plan	Hours
T12	agg_campaign_daily	Member 4	9-10 Oct	6
T14	agg_city_product_daily (rec_product cut first)	Member 4	10 Oct	4
T15	Batch runner script	Member 4	10-11 Oct	6
T16	Five SQL checks plus manifest check	Member 5	10-11 Oct	6
G3	Failure demo with Grafana	Member 5	11 Oct	4
T17	Samples, benchmark, README	Member 5	11-12 Oct	6
T18	Power BI model and pages	Member 1	11-12 Oct	10
DOC	Refresh PDF, deck, README screenshots	Member 5	12 Oct	6
T19	Customer 360 (STRETCH, cut first)	Member 4	12 Oct	4
M2-M4	Failure demo run, 1-2M benchmark; rehearsal; defense	All	12-13 Oct	-

Gold build slots: drop a file named `warehouse/gold/10_agg_campaign_daily.sql`, `11_agg_city_product_daily.sql`, `12_rec_product.sql` or `13_checks.sql` and the runner picks it up in order; update the test that lists empty slots in `tests/test_gold_build.py`.

T12. agg_campaign_daily: what to build and how do I know it is done?

- Inputs: `silver_campaign_snapshots` (latest pull per campaign and report date), `fact_attribution` (last_touch rows), `fact_orders`.
- Columns are fixed in `warehouse/tables.py` (key campaign_id, report_date, attribution_model = 'last_touch'): ad_platform, impressions, clicks, platform_conversions, spend; orders_placed, orders_delivered, orders_cancelled, orders_returned, orders_with_failed_attempt; revenue_placed, revenue_delivered; cancel_rate, failed_attempt_rate, return_rate, open_rate; new_customer_orders, aov, ctr, conversion_rate, cpa, cost_per_delivered_order, roas_placed, roas_delivered.
- Division by zero gives NULL (spend 0, orders 0). Delivered revenue is dated on the ORDER date. Two pulls of the same day never add up: use the latest.
- Done when: the SQL step passes the Gold-build lint tests, a Spark parity test matches a small Python reference or hand-computed rows (zero spend, zero orders, two pulls), the real workspace run is idempotent, and docs/metrics.md matches.

T14. agg_city_product_daily (and rec_product if time allows)?

- agg_city_product_daily (key city, product_id, date_key): sessions_viewed (**distinct sessions**, not customers), add_to_cart, units, revenue. City grain on purpose (no district).
- rec_product is cut first. If built: co-view recommendations; use a city for an anchor product only if at least 20 distinct sessions viewed it and the recommended product (labelled co-view, not purchase).
- Done when: parity or hand-computed tests, idempotent real run.

T15. Batch runner script?

- One command: publish_silver, then gold_build, then the checks (T16), then print a summary and exit non-zero on any failure. Python is primary; the Airflow DAG exists only for the ad report (decision date 10 Oct).
- Takes a store (no direct database imports). Writes the gold_build and publish heartbeats (already done in the two scripts).
- Done when: a fresh clone can run it end to end with the documented commands and it prints a success line with counts.

T16. Which five SQL checks?

- Silver quarantine and duplicate counts vs `noise_manifest.json` (tolerance 2%): add a Python check type that reads the manifest.

- No duplicate keys in any Gold table; no orphan keys; share of -1 keys reported.
- Revenue reconciliation: sum of order lines equals sum of orders (250,155.95 in the last real run).
- District hierarchy: zero violations in Silver; reported (non-failing) check for negative delivery_lead_hours and for orders with both a counted cancel and a later shipment.
- Row counts of Gold tables vs expectations. A failing check blocks the Power BI refresh and shows in Grafana.

G3. Failure demo with Grafana?

```
1. .\scripts\watch_demo.ps1 and open Grafana; the event line is moving.
2. docker stop fs-pipeline      # kill the Spark job: the line drops to zero,
                                # the 'no events' alert fires within 2-3 minutes
3. restart the pipeline window; the line recovers; the alert resolves
4. run the duplicate check on Silver: zero duplicate event_id (a script in scripts/)
```

Done when: the demo works twice in a row and a short script proves zero duplicates after the restart.

T17. Samples, benchmark, README?

- Export 10,000-row CSV samples per Gold table to `samples/` (no names or phones).
- Run the pipeline once at 1-2 million events and record time and throughput (10 million is a one-off benchmark only).
- README: architecture, quick start, screenshots of Grafana and Power BI.

T18. Power BI?

- Connect with the Databricks connector (server hostname and HTTP path from the SQL warehouse, a read-only token). Relationships on customer_key and product_key.
- Build Campaign and Funnel pages first; extra pages only if time remains.
- Done when: the numbers on the pages equal the SQL queries in Databricks for the same period.

DOC. What documents need refreshing?

The PDF (`docs/FunnelStream-Documentation.pdf`) and the deck are from 5 October: update table names (agg_/rec_), the scope table (Grafana is Required), Amendments C and D (returned status, COD, platform, district, tracking URL), attribution and live monitoring. This handoff PDF can be reused as the base.

What do we cut if we fall behind?

In order: T19 customer 360, then rec_product, then the SQL checks beyond the five key ones, then Power BI pages beyond Campaign and Funnel. **Never cut**: attribution, campaign summaries (T12), Grafana, and the failure demo (G3).

Part 4. Decisions, limits and troubleshooting

Decisions

Why not a real CDC pipeline (Debezium) for orders and deliveries?

Orders and deliveries already have their own topics, keyed by order id, at lower volume than app events. Real CDC needs Postgres logical replication plus Kafka Connect (about 1 GB more) and a rework of the simulator, Silver and the status model, with 5 days left. Decision: keep the event-sourced style. Production path to mention in the defense: orders in an operational database captured with Debezium. An optional Debezium-format simulation is possible after the required tasks.

Why Databricks for Gold and not Postgres?

Shared workspace for the team, SQL MERGE at scale, and a Power BI connector. Postgres holds only the small live tables. The Gold DDL also exists for Postgres as a fallback, but the Silver mirror tables and the ad report need Databricks.

Why last touch, with a 7-day fallback?

It is the simplest defensible model and it is stated on the table (attribution_model). first_touch rows are stored for comparison. The two models differ for a minority of orders (11 of 239 in the parity dataset).

Why does the live 'orders per minute' count purchase events?

One purchase per order in the simulator, with the same late and bot filters and event-time minute as the other panels. It can differ from silver_orders if a purchase event is missing or a session places two orders.

Known limits

- Synthetic data: distributions are documented assumptions, not measurements.
- Ramadan and Eid dates in dim_date are approximate (moon sighting).
- Late status rows older than the Gold window are picked up only by a wider window or --full.
- A summary key that vanishes on a widened date outside the window is not deleted.
- Anonymous pre-login sessions are not linked to customers after login.
- Gold SQL has run on Spark 3.5 locally and on the real Databricks workspace; new SQL must be run on both.
- Databricks Free Edition limits apply; the Postgres fallback exists for Gold only.
- The live-store write can lose pending minutes if the job stops while Postgres is down (Silver is unaffected).

Troubleshooting

Symptom	Cause and fix
docker: failed to connect to the docker API	Docker Desktop is not running. Start it and wait a minute.
Exit code 137 from a Spark container	Out of memory. Stop other containers (do not run tests during the live demo) and rerun.
Missing expression after unary operator '!'	The ! prefix works only inside Claude Code. In PowerShell run the command without it.
Silver shows no new rows after a second run	Same --seed: the ids repeat and Silver drops them as duplicates. Use a new --seed.
Counts too high after a rerun	Kafka still holds old messages. Use docker compose down, delete data\lake and data\checkpoints, start again.
Databricks mirror counts higher than local	The mirror only adds rows. Run publish_silver.py --reset --yes, then publish again.
Grafana panels empty	No recent data: run the generator in --live mode. Check docker compose ps for postgres (healthy) and grafana.
Grafana cannot connect to the database	GRAFANA_DB_PASSWORD missing from .env when apply_live_ddl ran. Set it and rerun docker compose run --rm spark python scripts/apply_live_ddl.py.
Gold tables empty	Run gold_build after publish_silver. fact_* need the Silver mirror filled first.

Symptom	Cause and fix
A Spark test fails only today	Check for a hard-coded date against the default 7-day window; pass an explicit --from-date/--to-date window in the test.
Port 9092 in use	Another Kafka. Stop it or docker compose down. Kafka in a browser shows nothing: it is not a web page.

Appendix

Repository map

Folder or file	What is inside
simulator/, scripts/run_simulator.py	Event generator, noise, campaigns, publisher
mock_apis/ad_api.py	Fake Meta/Google ads API (port 8000)
contracts/	Event schemas, validator, normalizers, geo.py (city and district hierarchy)
kafka/create_topics.py	Creates the three topics
streaming/jobs/	bronze_ingest.py, silver_clean.py, pipeline.py (both in one process)
streaming/silver/, streaming/common/	Silver rules and schemas; session, status rank, bot rule
streaming/gold/	Python reference builders (orders, funnel, attribution, live metrics, bots)
warehouse/	store.py (only database layer), tables.py (table spec), gold_build.py, gold/*.sql, live_ddl.sql, live_store.py, heartbeat.py, publish.py
scripts/	apply_ddl, apply_live_ddl, publish_silver, check_databricks, seed_silver_reference, show_live, alert_sink, watch_demo, stop_demo
batch/, airflow/dags/	Ad report logic and its Airflow DAG
monitoring/grafana/	Provisioned datasource, dashboard JSON and alert rules
tests/	Host tests, Spark tests (Docker), integration tests (compose Postgres)
docs/	spec, plan, RUNNING.md, metrics.md, sprint sheet v3, PDFs, this handoff

Vocabulary

Term	Meaning
Bronze / Silver / Gold	Raw / cleaned / business-ready layers
Quarantine	Table of rejected rows with an error code (not a dead-letter topic)
Idempotent	Running twice gives the same result; reruns never duplicate
MERGE	Upsert statement used for every Silver and Gold write
SCD2	Dimension that keeps history with valid_from, valid_to, is_current
Unknown member	Dimension row with key -1 for facts whose dimension row does not exist yet
Manifest	noise_manifest.json: exact count of every injected defect
Last touch	Order credited to the latest campaign click before it
COD	Cash on delivery (skips payment_authorised)
District	Official subdivision (qism or hay) of a city

Generated from `docs/handoff/build_handoff_pdf.py`. To update: edit the script, run it, commit the PDF.